

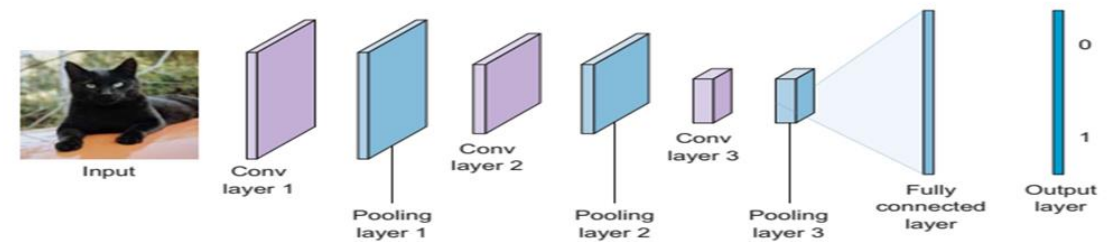
Deep Learning

Lecture 6

What we did?

- We build Our model
- ```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential()
model.add(Conv2D(filters=16, kernel_size=2, padding='same',
activation='relu', input_shape=(32, 32, 3)))
model.add(MaxPooling2D(pool_size=2))
model.add(Conv2D(filters=32, kernel_size=2, padding='same',
activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Conv2D(filters=64, kernel_size=2, padding='same',
activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Dropout(0.3))
model.add(Flatten())
model.add(Dense(500, activation='relu'))
model.add(Dropout(0.4))
model.add(Dense(10, activation='softmax'))
model.summary()
```



# What next?

## COMPILE THE MODEL

The last step before training our model is to define three more hyperparameters loss function, an optimizer, and metrics to monitor during training and testing:

- Loss function—How the network will be able to measure its performance on the training data.
- Optimizer—The mechanism that the network will use to optimize its parameters (weights and biases) to yield the minimum loss value. It is usually one of the variants of stochastic gradient descent.
- Metrics—List of metrics to be evaluated by the model during training and testing. Typically we use `metrics=['accuracy']` for classification.
- `model.compile(loss='categorical_crossentropy', optimizer=SGD, metrics=['accuracy'])`

- **TRAIN THE MODEL**

- `model.fit(x_train, y_train, batch_size=32, epochs=100, validation_data=(x_valid, y_valid), verbose=2, shuffle=True)`

- 

what is the batch\_size and epochs hyper-parameter?

- **EVALUATE THE MODEL**

- `score = model.evaluate(x_test, y_test, verbose=0)`  
`print('\n', 'Test accuracy:', score[1])`

# Defining performance metrics

- Performance metrics allow us to evaluate our system. When we develop a model, we want to find out how well it is working. The simplest way to measure the “goodness” of our model is by measuring its accuracy. The accuracy metric measures how many times our model made the correct prediction. So, if we test the model with 100 input samples, and it made the correct prediction 90 times, this means the model is 90% accurate.
- Confusion matrix :a table that describes the performance of a classification model.
  - True Positive (TP): denote the instances that the model correctly predicts in the positive class.
  - False Positive (FP): denote the instances that the model incorrectly predicts in the positive class.
  - True Negative (TN): denote the instances that the model correctly predicts in the negative class.
  - False Negative (FN): denote the instances that the model incorrectly predicts in the negative class.
- Accuracy = correct prediction / total number
 
$$Accuracy (AC) = \frac{TP + TN}{TP + TN + FP + FN}$$

|                 |          | True Class |          |
|-----------------|----------|------------|----------|
|                 |          | Positive   | Negative |
| Predicted Class | Positive | TP         | FP       |
|                 | Negative | FN         | TN       |

If our dataset labelled as is sick patient and well patient :

- Recall (also known as sensitivity) tells us how many of the sick patients our model incorrectly diagnosed as well. In other words, how many times did the model incorrectly diagnose a sick patient as negative (false negative, FN)? Recall is calculated by the following equation:

$$\text{Recall} = \text{True Positive Rate (TPR)} = \frac{TP}{TP + FN}$$

- *Precision* (also known as *specificity*) is the opposite of recall. It tells us how many of the well patients our model incorrectly diagnosed as sick. In other words, how many times did the model *incorrectly* diagnose a well patient as positive (false positive, FP)? Precision is calculated by the following equation:

$$\text{precision} = \frac{TP}{TP + FP}$$

- Identifying an appropriate metric

It is important to note that although in the example of health diagnostics we decided that recall is a better metric, other use cases require different metrics, like precision. To identify the most appropriate metric for your problem, ask yourself which of the two possible false predictions is more consequential: false positive or false negative. If your answer is FP, then you are looking for precision. If FN is more significant, then recall is your answer.

- In some applications, you might care about both precision and recall at the same time. That's called an F-score, as explained next.

### ***F-score***

In many cases, we want to summarize the performance of a classifier with a single metric that represents both recall and precision. To do so, we can convert precision ( $p$ ) and recall ( $r$ ) into a single F-score metric. In mathematics, this is called the *harmonic mean* of  $p$  and  $r$ :

$$\text{F-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

|              | Precision | Recall | F-score |
|--------------|-----------|--------|---------|
| Classifier A | 95%       | 90%    | 92.4%   |
| Classifier B | 98%       | 85%    | 91%     |

# Getting your data ready for training

## Splitting your data for train/validation/test

- When we train a ML model, we split the data into train and test datasets .
- We use the training dataset to train the model and update the weights, and then we evaluate the model against the test dataset that it hasn't seen before.
- The golden rule here is this: *never use the test data for training*.
  - The reason we should never show the test samples to the model while training is to make sure the model is not cheating. We show the model the training samples to learn their features, and then we test how it generalizes on a dataset that it has never seen, to get an unbiased evaluation of its performance.

- **WHAT IS THE VALIDATION DATASET?**

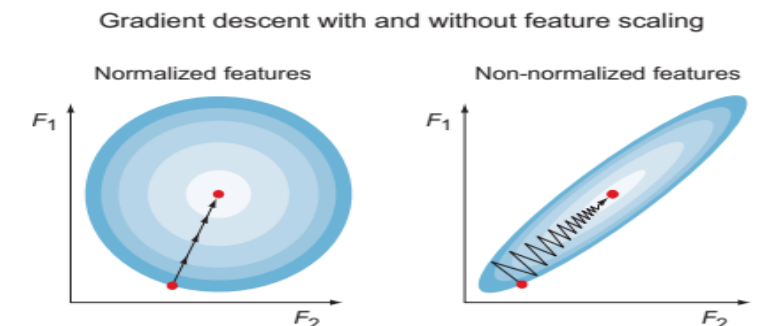
After each epoch during the training process, we need to evaluate the model's accuracy and error to see how it is performing and tune its parameters. If we use the test dataset to evaluate the model during training, we will break our golden rule of never using the testing data during training. The test data is only used to evaluate the final performance of the model *after* training is complete. So we make an additional split called a *validation dataset* to evaluate and tune parameters *during* training

- WHAT IS A GOOD TRAIN/VALIDATION/TEST DATA SPLIT?

Traditionally, an 80/20 or 70/30 split between train and test datasets is used in ML projects. When we add the validation dataset, we went with 60/20/20 or 70/15/15. But that was back when an entire dataset was just tens of thousands of samples.

- huge amount of data we have now, sometimes 1% for both the validation and the test set is enough. For example, if our dataset contains 1 million samples, 10,000 samples is very reasonable for each of the test and validation sets.
- **Data preprocessing:**
  - IMAGE GRAYSCALING
  - IMAGE RESIZING
  - DATA NORMALIZATION : is the process of rescaling your data to ensure that each input feature (pixel, in the image case) has a similar data distribution.

Normalized features help the GD algorithm go straight forward toward the minimum error, thereby reaching it quickly (left). With non-normalized features, the GD oscillates toward the direction of the minimum error and reaches the minimum more slowly (right).



# Evaluating the model and interpreting its performance

- *Diagnosing overfitting and under fitting*
- After running your experiment, you want to observe its performance, determine if bottlenecks are impacting its performance, and look for indicators of areas you need to improve.
- The main cause of poor performance in ML is either overfitting or underfitting the training dataset.
  - Underfitting means the model is *too simple*: it fails to learn the training data, so it performs poorly on the training data.
  - Overfitting is when the model is *too complex* for the problem at hand. Instead of learning features that fit the training data, it actually memorizes the training data. So it performs very well on the training data, but it fails to *generalize* when tested with new data that it hasn't seen before.
- **The Goal** is to build a model that is *just right* for the data: not too complex, causing overfit, or too simple, causing underfit.

- To diagnose underfitting and overfitting, the two values to focus on while training are the training error and the validation error:
- If the model is doing very well on the training set but relatively poorly on the validation set, then it is overfitting.
- For example, if train\_error is 1% and val\_error is 10%, it looks like the model has memorized the training dataset but is failing to generalize on the validation set.
- In this case, you might consider tuning your hyperparameters to avoid overfitting and iteratively train, test, and evaluate until you achieve an acceptable performance.
- If the model is performing poorly on the training set, then it is underfitting .
- For example, if the train\_error is 14% and val\_error is 15%,
- In this case, the model might be too simple and is failing to learn the training set. You might want to consider adding more hidden layers or training longer (more epochs), or try different neural network architectures. Also In this case, collecting more data could be effective.

# Discussion

- What is the different between *Parameters* and *hyperparameters* ?

- *Network architecture*

First, let's talk about the hyperparameters that define the neural network architecture:

- ☐ Number of hidden layers (representing the network depth)
- ☐ Number of neurons in each layer, also known as hidden units (representing the network width)
- ☐ Activation functions

- **DEPTH AND WIDTH OF THE NEURAL NETWORK**

too much learning capacity (too many hidden units), it might tend to overfit the data and memorize the training set. If your model is overfitting, you might want to decrease the number of hidden units.

- Generally, it is good to add hidden neurons until the validation error no longer improves.
- Having a small number of units may lead to underfitting, while having more units is usually not harmful, with appropriate regularization (like dropout )

- **ACTIVATION TYPE**

Activation functions introduce nonlinearity to our neurons. Without activations, our neurons would pass linear combinations (weighted sums) to each other and not solve any nonlinear problems.

- ReLU and its variations (like Leaky ReLU) perform the best in hidden layers. And in the output layer, it is very common to use the softmax function for classification problems

- The more neurons in your network, the more parameters the network has to optimize, and effect on computational complexity.
- To reduce the number of training parameters, you can do one of the following:
  - - Reduce the depth and width of the network (hidden layers and units). This will reduce the number of training parameters and, hence, reduce the neural network complexity.
    - Add pooling layers, or tweak the strides and padding of the convolutional layers to reduce the feature map dimensions. This will lower the number of parameters.

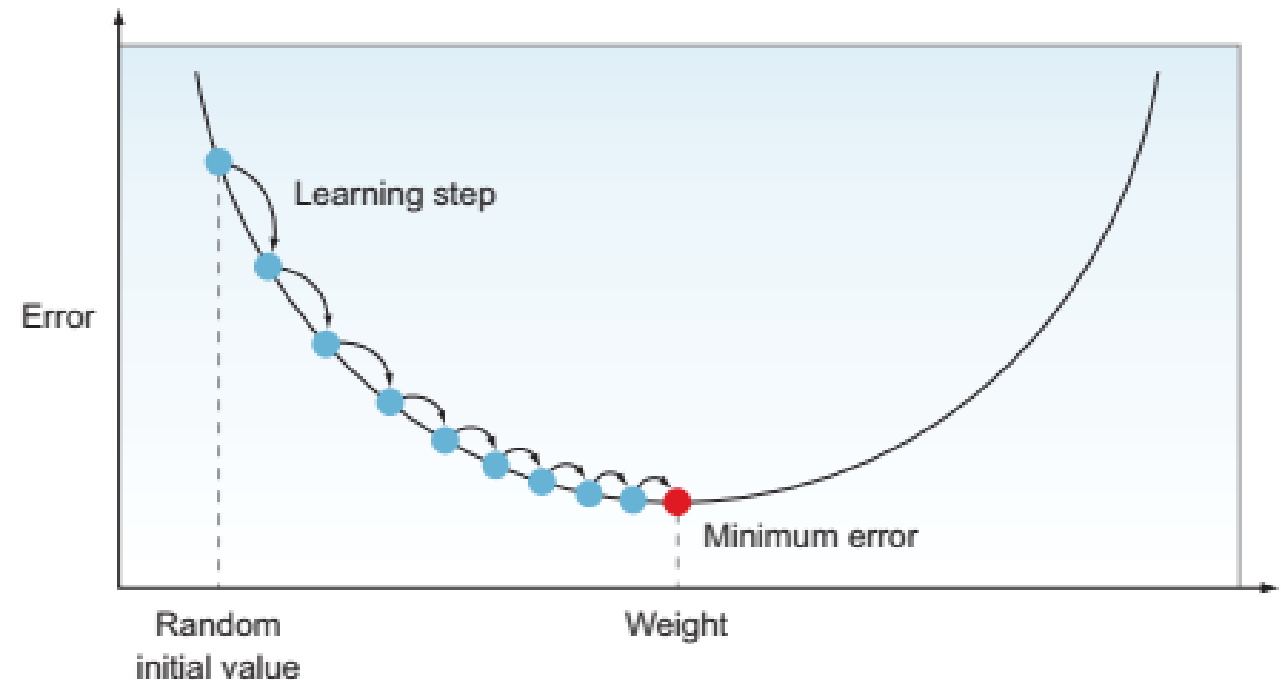
# Learning and optimization

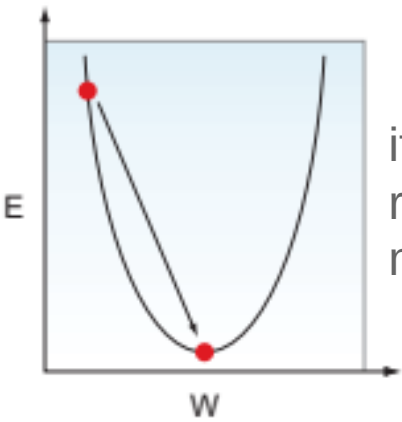
## Learning rate and decay schedule

The learning rate is the single most important hyperparameter, and one should always make sure that it has been tuned. If there is only time to optimize one hyperparameter, then this is the hyperparameter that is worth tuning .

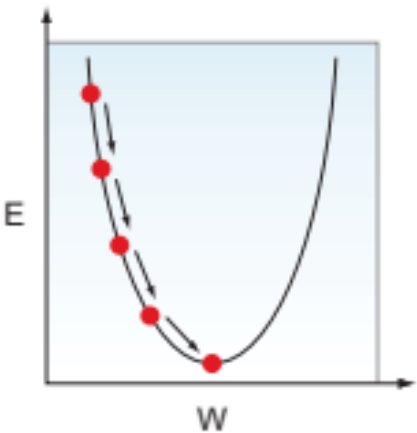
The GD optimizer searches for the optimal values of weights that yield the lowest error possible. When setting up our optimizer, we need to define the step size that it takes when it descends the error mountain. This step size is the learning rate.

The GD calculates the gradient to find the direction

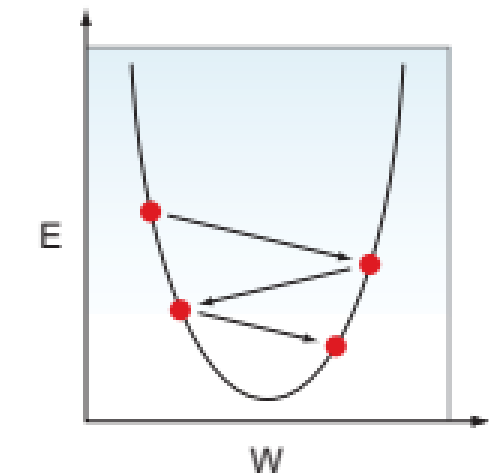




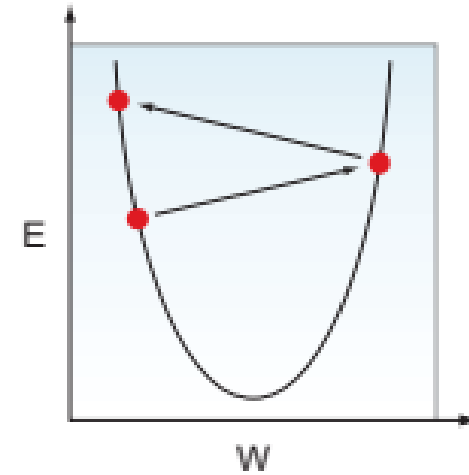
if we make a *miraculously* correct choice of the learning rate value, we land on the best weight value that minimizes the error in only one step



A learning rate *smaller* than the ideal lr value: the model takes smaller steps down the error curve.



A learning rate *larger* than the ideal lr value: the optimizer overshoot the optimal weight value.



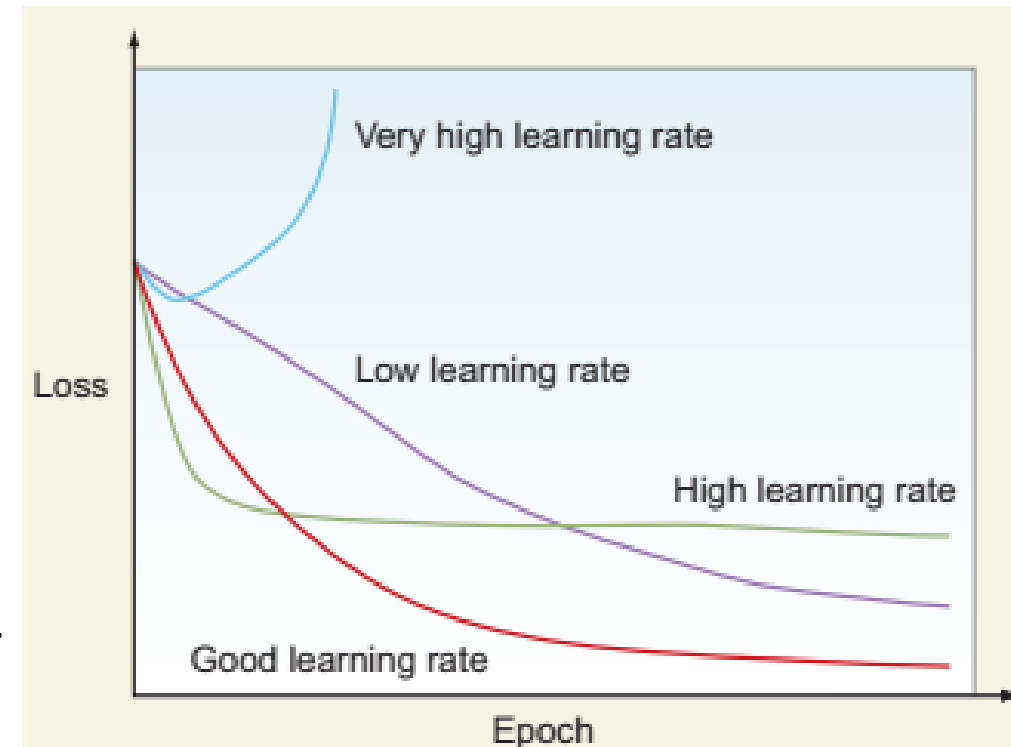
A learning rate much larger than the ideal lr value: the optimizer gets farther from the min error.

# Too-high vs. too-low learning rate

- Setting the learning rate high or low is a trade-off between the optimizer speed versus performance.
- Too-low lr requires many epochs to converge, often too many. Theoretically, if the learning rate is too small, the algorithm is guaranteed to eventually converge if kept running for infinity time.
- Too-high lr might get us to a lower error value faster because we take bigger steps down the error curve, but there is a better chance that the algorithm will oscillate and diverge away from the minimum value. So, ideally, we want to pick the lr that is just right (optimal): it swiftly reaches the minimum point without being so big that it might diverge.

When plotting the loss value against the number of training iterations (epochs), you will notice the following:

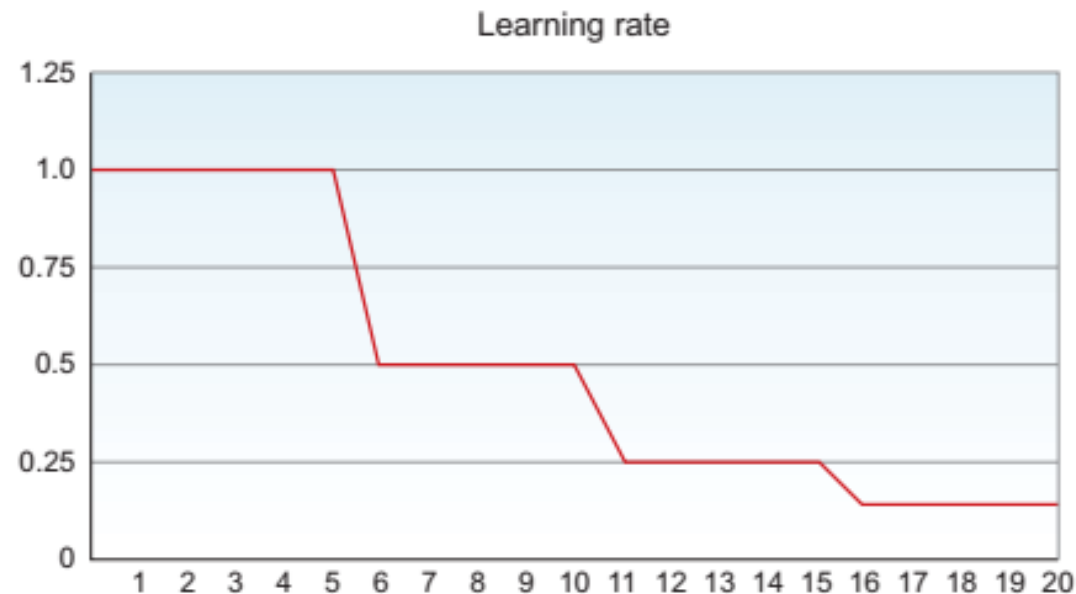
- Much smaller lr—The loss keeps decreasing but needs a lot more time to converge.
- Larger lr—The loss achieves a better value than what we started with, but is still far from optimal.
- Much larger lr—The loss might initially decrease, but it starts to increase as the weight values get farther and farther away from the optimal values.
- Good lr—The loss decreases consistently until it reaches the minimum possible value.



The way to debug this is to look at the validation loss values in the training verbose:

- If val\_loss decreases after each step, that's good. Keep training until it stops improving.
- If training is complete and val\_loss is still decreasing, then maybe the learning rate was so small that it didn't converge yet. In this case, you can do one of two things:
  - Train again with the same learning rate but with more training iterations (epochs) to give the optimizer more time to converge.
  - Increase the lr value a little and train again
- If val\_loss starts to increase or oscillate up and down, then the learning rate is too high and you need to decrease its value.

- Another way to go about tuning your learning rate is to set a learning rate decay: a method by which the learning rate changes *during* training. It often performs better than a static value, and drastically reduces the time required to get optimal results.
- The decay rate tells our network to automatically decrease the lr throughout the training process.
- For example, we can decrease the lr by a constant value of ( $x$ ) for each ( $n$ ) number of steps. This way, we can start with the higher value to take bigger steps toward the minimum, and then gradually decrease the learning rate every ( $n$ ) epochs to avoid overshooting the ideal lr.
- One way to accomplish this is by reducing the learning rate linearly (*linear decay*).



- Another way is to decrease the lr exponentially (*exponential decay*).
- For example, you can multiply it by 0.1 every eight epochs. Clearly, the network will converge a lot slower than with linear decay, but it will eventually converge.

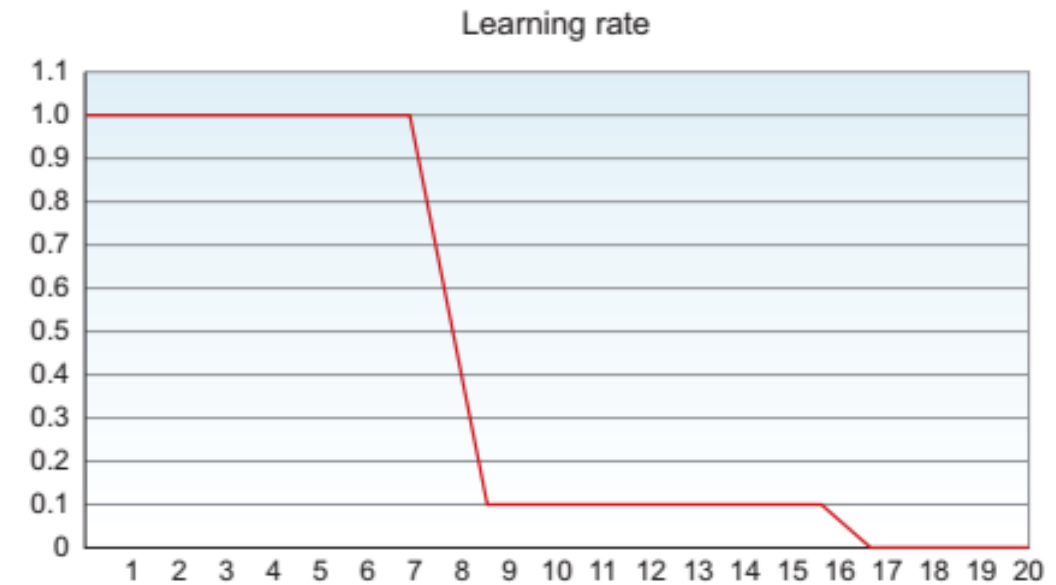
Other clever learning algorithms have an adaptive learning rate (*adaptive learning*).

These algorithms use a heuristic approach that automatically updates the lr when the training stops.

This means not only decreasing the lr when needed, but also increasing it when improvements are too slow (too-small lr).

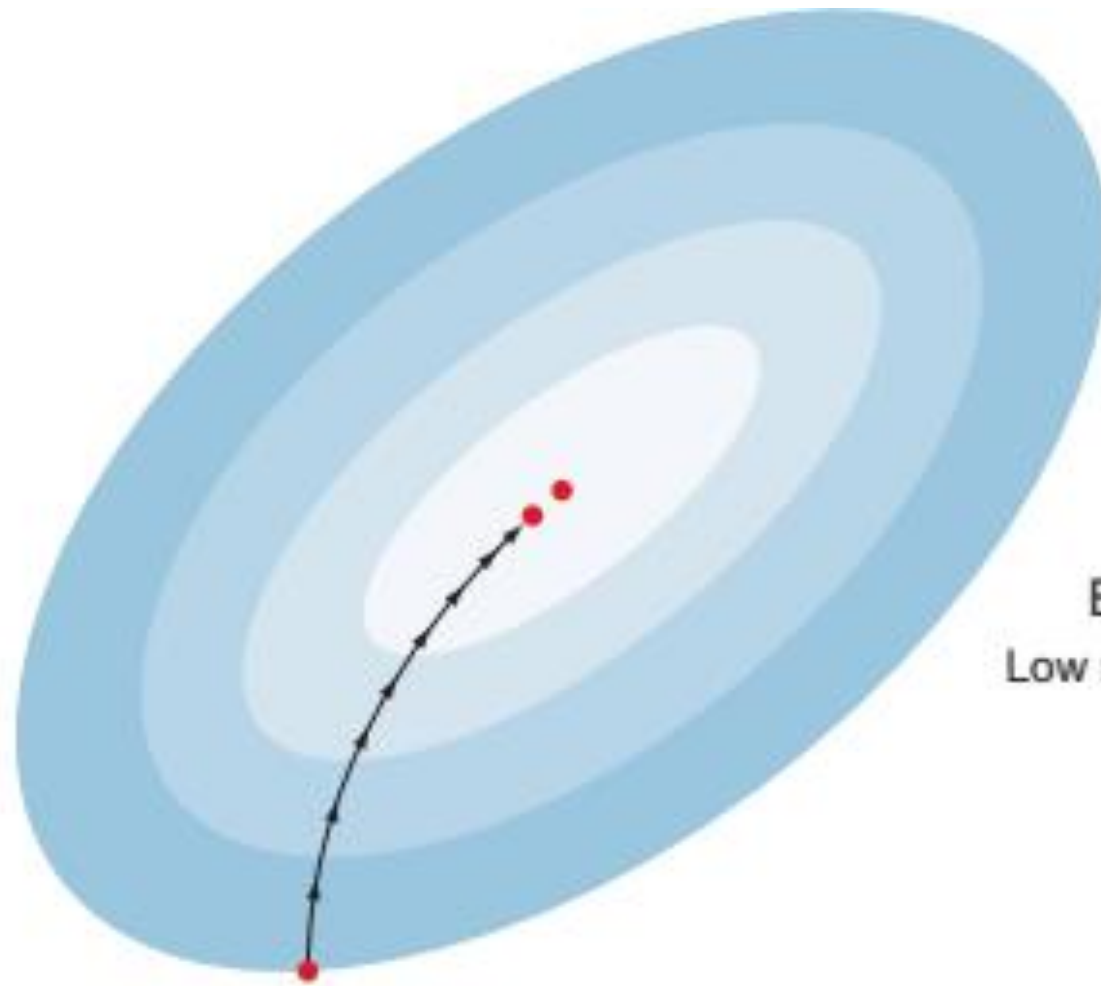
Adaptive learning usually works better than other learning rate-setting strategies.

**Adam** and **Adagrad** are examples of adaptive learning optimizers, we will study them in next Saturday class



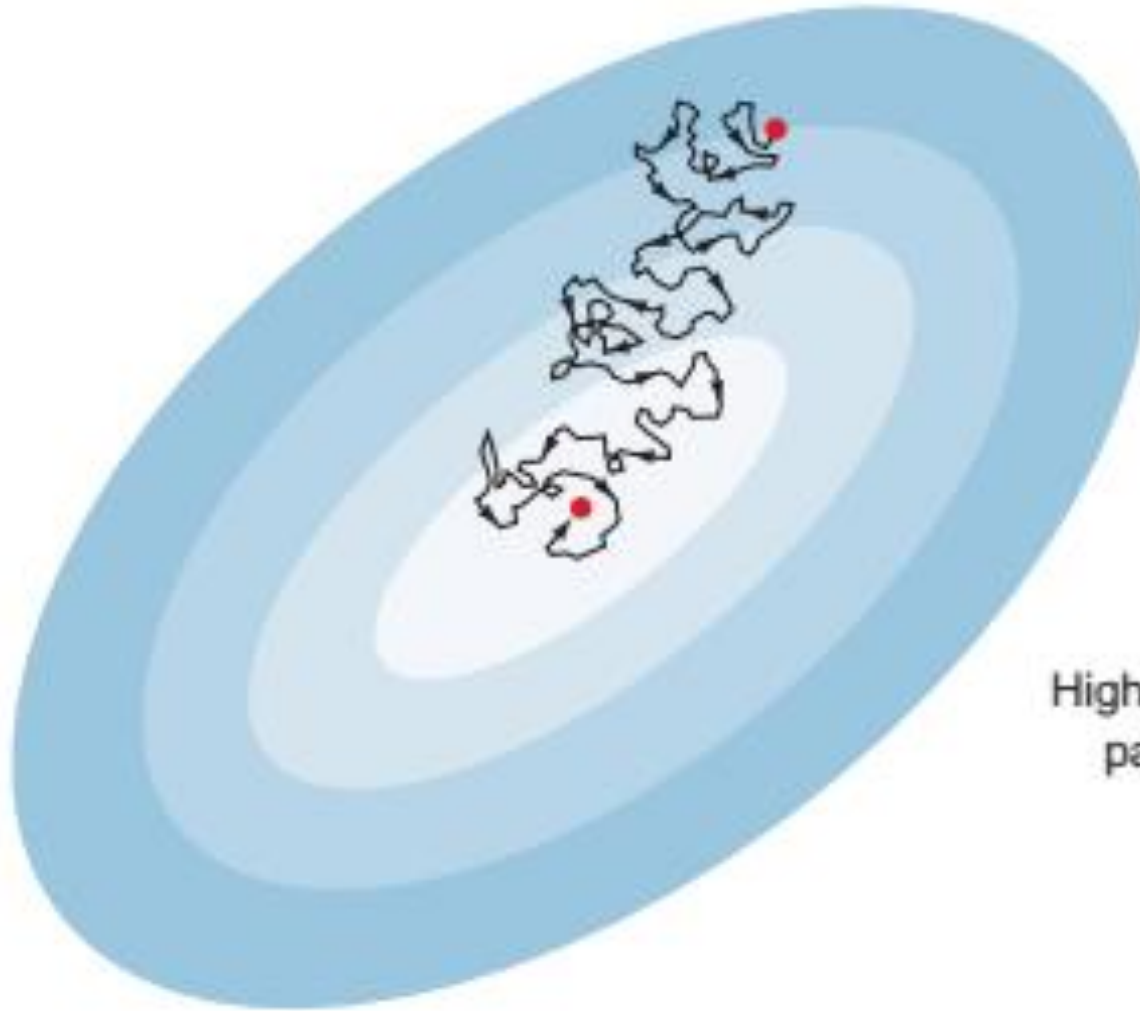
# Mini-batch size

- Mini-batch size is another hyperparameter that you need to set and tune in the optimizer algorithm. The `batch_size` hyperparameter has a big effect on resource requirements of the training process and speed.
- ***Batch gradient descent (BGD)***—We feed the entire dataset to the network all at once, apply the feedforward process, calculate the error, calculate the gradient, and backpropagate to update the weights.
- The optimizer calculates the gradient by looking at the error generated after it sees *all* the training data, and the weights are updated *only once* after each epoch. So, in this case, the mini-batch size equals the entire training dataset.
- The main advantage of BGD is that it has relatively low noise and bigger steps toward the minimum.
- The main disadvantage is that it can take too long to process the entire training dataset at each step, especially when training on big data. BGD also requires a huge amount of memory for training large datasets, which might not be available. BGD might be a good option if you are training on a small dataset



**Batch gradient descent (BGD)**  
Low noise on its path to the minimum error

- **Stochastic gradient descent (SGD)**—Also called *online learning*. We feed the network a single instance of the training data at a time and use this one instance to do the forward pass, calculate error, calculate the gradient, and backpropagate to update the weights. In SGD, the weights are updated after it sees each single instance (as opposed to processing the entire dataset before each step for BGD).
- SGD can be extremely noisy as it oscillates on its way to the **global minimum** because it takes a step down after each single instance, which could sometimes be in the wrong direction. This noise can be reduced by using a smaller learning rate, so, on average, it takes you in a good direction and almost always performs better than BGD.
- With SGD you get to make progress quickly and usually reach very close to the global minimum.
- The main disadvantage is that by calculating the GD for one instance at a time, you lose the speed gain that comes with matrix multiplication in the training calculations.



### Stochastic (GD)

High noise and oscillates on its path to the minimum error

- **Mini-batch gradient descent (MB-GD)**—A compromise between batch and stochastic GD. Instead of computing the gradient from one sample (SGD) or all training samples (BGD), we divide the training sample into mini-batches to compute the gradient from. This way, we can take advantage of matrix multiplication for faster training and start making progress instead of having to wait to train the entire training set.
- **Guidelines for choosing mini-batch size**  
First, if you have a small dataset (around less than 2,000), you might be better off using BGD. You can train the entire dataset quite fast.
- For large datasets, you can use a scale of mini-batch size values. A typical starting value for the mini-batch is 64 or 128. You can then tune it up and down on this scale: 32, 64, 128, 256, 512, 1024, and keep doubling it as needed to speed up training. But make sure that your mini-batch size fits in your CPU/GPU memory.
- Mini-batch sizes of 1024 and larger are possible but quite rare. A larger mini-batch size allows a computational boost that uses matrix multiplication in the training calculations. But that comes at the expense of needing more memory for the training process and generally more computational resources.

- figure shows the relationship between batch size, computational resources, and number of epochs needed for neural network training

